

Visitor Counting Console

Feature list and technical specification

30 September 2026

Contents

1	What it is	1
2	Feature list	2
2.1	Counting	2
2.2	Demographics	2
2.3	Counting areas	2
2.4	Cameras	3
2.5	Capacity and alerts	3
2.6	Recording and history	3
2.7	Operations	4
3	Technical specification	4
3.1	Pipeline	4
3.2	Models	4
3.3	Key parameters	5
3.4	Interfaces	5
3.5	Data outputs	6
3.6	Configuration	6
3.7	Platforms	6
3.8	Performance	6
3.9	Accuracy	7
3.10	Known limits	7

1 What it is

A visitor counting system that runs on the CCTV a venue already has. Each camera gets a trigger line drawn across its entrance; a person is counted when the head point of their track crosses that line, and a zero-shot classifier labels them male or female and adult or child. There is no turnstile, no new hardware and no app on anyone's phone.

One machine with a GPU runs everything: video decoding, detection, tracking, classification, counting, the web console and the history log. There is no database, no cloud service and no per-site model training.

Two front ends sit on one engine:

- **Browser console** (`webapp.py`) — a crowd-management panel and one panel per camera, served over HTTP. This is the primary interface.
- **OpenCV windows** (`app.py`) — a 2x2 kiosk grid for a screen with no browser.

2 Feature list

2.1 Counting

- **Trigger line per camera**, drawn on the live picture and dragged into place. Stored in normalised coordinates, so it survives a change of stream resolution or a move to a different machine.
- **Head-point crossing**. The top-centre of the body box (8 % down) is what crosses the line, or the mean of the facial keypoints when a pose model is used. Feet-based crossing is much noisier on an angled camera.
- **Direction**. Crossing toward the marked *IN* side is an entry, the reverse an exit. The direction is reversed from the page with one button.
- **Segment tolerance**. A crossing still counts for 8 % of the line’s length past each end, so a line drawn slightly short of a doorway still catches its edges. The overlay draws that reach as a dashed extension, because an invisible tolerance is indistinguishable from a bug when the numbers look wrong.
- **Edge warning**. Saving a line within 4 % of the frame edge warns: a person can cross it before the tracker has established them, which miscounts.
- **Anti-double-counting**. One event per side transition per track; a hysteresis dead zone around the line plus a multi-frame confirmation absorb detector jitter; a brand-new track crossing at the same spot immediately after another one is treated as a tracker ID switch and ignored.
- **Two distinct figures**. *Visitors today* counts entries only and never falls. *Current occupancy* is net and falls again when people leave. Someone who walks in and out has still visited, so conflating the two misleads.

2.2 Demographics

- **Zero-shot classification** of gender and age group from the person crop. No per-site training and no face required, which matters on overhead cameras.
- **Split voting strategy**. Gender is decided from the last three observations plus one taken at the instant of crossing, counted double. Age averages every observation. The two attributes behave differently: the gender read swings early in a track and settles at the line, while “child” is noisy per frame and smooths out.
- **Crop budget**. Classification runs every 15 processed frames per track, at most 6 crops per camera per frame, least-observed track first. People below the minimum crop height are counted but not classified until they come closer.
- **Honest unknowns**. A track the classifier has not decided is reported as unknown rather than guessed, so the male/female and adult/child splits always reconcile to the total.
- **Late revision**. A crossing is decided the moment the head crosses, when the person may have only just entered frame. Once the track has been watched longer the vote is better informed, so the count is moved if the decision changed, and the revision is logged.

2.3 Counting areas

A rectangle drawn on the picture that limits what a camera does. Each area carries a **role**, chosen when it is drawn, because restricting what is *tracked* and restricting what is *classified* are different needs:

role	effect	may the line sit outside it?
count only inside	only people inside are tracked at all; outside is dimmed	no — saving warns

role	effect	may the line sit outside it?
ignore inside	people inside are skipped: a window, a mirror, a poster, a screen	yes
classify only inside	everyone is counted, but only those inside are sexed and aged	yes

Count only inside must contain the whole trigger line with room on both sides. A person is only tracked once inside the box, so one approaching from outside is first seen already at the line and is seeded on the wrong side. The console warns when a box is saved that does not.

Classify only inside is the accuracy lever on a wide scene: put it where people are large and well lit and the crop budget stops being spent on distant figures. Tracks outside it still count, they simply read as unknown.

An area also reports **its own population** — who is standing in it right now, split male/female and adult/child, with an unknown count so the parts sum to the total. This owes nothing to the trigger line, so a box reads correctly from the first frame rather than starting at zero.

Minimum area size is 5 % of the frame on each side; smaller is refused as a stray click.

2.4 Cameras

- **Up to six cameras**, added and removed on the page while running. The grid re-lays itself out around the number present.
- **RTSP, HTTP or a video file** per camera. Credentials with @, :, \$, %, & or spaces are percent-encoded automatically on the way to the decoder, so an operator pastes what the camera’s own page shows. /, ? and # remain ambiguous in a URL and are rejected with instructions rather than silently failing.
- **Paired feeds**. A camera slot can hold both a live RTSP URL and a saved clip; one click, or the v key, swaps the panel between them, and **Shift+V** swaps every panel at once. Demonstrating on a clip is usually more convincing than a live camera pointed at an empty corridor.
- **Offline handling**. An unreachable camera reports **OFFLINE** and hides its figures rather than showing frozen ones. Opening a stream is capped by a timeout so one dead camera never blocks the console, and reconnection is automatic.
- **Per-camera naming** that persists, so the panel reads “Main entrance” rather than an RTSP URL.

2.5 Capacity and alerts

- **Site capacity** with a warning threshold. Crossing the threshold raises an amber band; reaching capacity raises a red banner that names the entrance whose entry tipped the count, marks every camera frame, and records a clip automatically.
- **Per-area capacity**. Any single entrance can carry its own limit and raise its own alarm independently of the site total.
- **Alert list** covering over capacity, approaching capacity, camera offline, file ended, missing trigger line, calibration in progress, paused, and child entered. Alerts are acknowledged individually or all at once, and re-raise if the condition returns.

2.6 Recording and history

- **Event recording**. One button saves the preceding 10 s and the following 20 s of that camera as an MP4 plus a snapshot, with an operator note. Over capacity does the same automatically.

- **Crossing log** — one row per counted crossing, with the probabilities and observation count behind the decision, so any number can be audited back to the frames that produced it.
- **Daily history** — hourly rows while open, a closing row at closing time, plus corrections and recorded events.
- **Head-count correction.** An operator can set the true occupancy from a manual count at the door; the correction is logged rather than applied silently.

2.7 Operations

- **Building day set-up** asks once per day for date, opening and closing times, capacity, warning threshold and starting occupancy. The count resets itself at opening time.
- **Opening occupancy defaults to zero** deliberately. A non-zero baseline has to be split by assumption — 50/50 male/female, 80/20 adult/child — and that guess would propagate into every demographic figure. It is set only when a real head count is known.
- **Runtime configuration is written by the application** and kept out of source control, seeded from committed templates. A deployment machine's own cameras, lines and hours can never collide with a source update.
- **Pause and resume**, and a manual counter reset, from the page footer.

3 Technical specification

3.1 Pipeline

RTSP / HTTP / file

- > threaded decode, one thread per source, frame-strided
- > YOLO11 person detection, one batched pass across all cameras
- > ByteTrack, one tracker instance per camera
- > head point vs trigger line -> entry (+1) / exit (-1)
- > SigLIP 2 zero-shot gender and adult/child on selected crops
- > counters, alert engine, CSV logs
- > MJPEG overlay stream + JSON state over WebSocket

Decoding, inference and rendering run in separate threads so decode cost overlaps with inference.

3.2 Models

role	model	licence
person detection	Ultralytics YOLO11s (yolo11s.pt, ~18 MB)	AGPL-3.0
demographics	google/siglip2-base-patch16-256 (~1.4 GB)	Apache-2.0
tracking	ByteTrack (configuration in config/bytetrack.yaml)	—

Both model files download once at set-up and are then used offline. The classifier sits behind a single-method interface, so replacing it with a purpose-trained attribute model is a contained change.

Licensing note. The detector is AGPL-3.0 while this codebase is not. Offering this as a network service to third parties may oblige disclosure of that service's source under the AGPL, or a commercial licence from Ultralytics. Only `humanmonitor/detector.py` imports it.

3.3 Key parameters

parameter	default	meaning
--classify-every	15	processed frames between re-classifications of a track
--max-crops	6	crops classified per camera per frame
--min-crop-height	48 px	below this a person is counted but not classified
--initial-count	0	people assumed already inside at start-up
--child-thresh	0.6	probability required to count a child
--conf	0.25	detector confidence
--max-cameras	6	upper bound of camera panels
--port	8780	console port
segment tolerance	8 %	crossing reach past each line end
edge margin	4 %	proximity to the frame edge that triggers a warning
recent observations	3	looks kept for the gender vote
crossing weight	2.0	weight of the look taken at the line
minimum area	5 %	smallest side of a counting area

3.4 Interfaces

HTTP and WebSocket, 24 endpoints:

method	path	purpose
GET	/	the console
GET	/api/state	full state snapshot
GET	/api/history	history rows
GET	/stream/{cid}	MJPEG stream with overlays
GET	/snapshot/{cid}.jpg	single annotated frame
WS	/ws	state push, about 4 Hz
POST	/api/cameras	add a panel
POST/DELETE	/api/cameras/{cid}/source	set or clear a feed
POST/DELETE	/api/cameras/{cid}/line	set or clear a trigger line
POST/DELETE	/api/cameras/{cid}/region	set or clear a counting area
POST	/api/cameras/{cid}/flip	reverse the entry direction
POST	/api/cameras/{cid}/swap	swap to the paired feed
POST	/api/cameras/{cid}/record	record an event clip
POST	/api/cameras/{cid}/editing	pause counting while calibrating
POST	/api/site, /api/setup	building settings
POST	/api/occupancy	head-count correction
POST	/api/reset, /api/pause	counters and pipeline
POST	/api/alerts/ack	acknowledge alerts
DELETE	/api/cameras/{cid}	remove a panel

The MJPEG stream is encoded once per camera and shared between viewers, so additional browsers cost bandwidth rather than GPU.

3.5 Data outputs

output/events.csv — one row per counted crossing:

timestamp, camera_id, camera_name, source, track_id, direction,
gender, age, p_male, p_child, n_obs, event

output/history.csv — hourly, closing, correction and event rows:

timestamp, kind, date, occupancy, entries, exits, male, female, adult, child,
visitors, v_male, v_female, v_adult, v_child, capacity, pct, cameras, note, clip
male..child are net inside; visitors and v_* are cumulative entries.

output/events/ — recorded MP4 clips and JPEG snapshots, roughly 10 MB per clip.

An optional per-frame track log is available for evaluation work, carrying box coordinates, detector confidence and the running probabilities.

3.6 Configuration

Three JSON files, written by the application and excluded from source control, each seeded from a committed template:

file	holds
config/sources.json	the live (RTSP) and video (file) source sets; slot <i>i</i> in one pairs with slot <i>i</i> in the other
config/lines.json	trigger lines and counting areas per source, in normalised coordinates
config/site.json	site name, operating day, hours, capacity, warning threshold, starting occupancy

Keeping them untracked is what stops a source update on a deployment machine from colliding with that machine’s own calibration, and keeps camera credentials out of the repository.

3.7 Platforms

platform	acceleration	set-up
Ubuntu 22.04 / 24.04	NVIDIA CUDA	<code>./setup_ubuntu.sh</code>
Windows 10 / 11	NVIDIA CUDA	<code>setup_windows.bat</code>
macOS (Apple Silicon)	Apple MPS	<code>./setup_mac.sh</code>
Ubuntu + Docker	NVIDIA CUDA	<code>docker compose up --build</code>

Python 3.12. Device selection is automatic: CUDA, then Apple MPS, then CPU. On CUDA the detector and classifier run in FP16. A TensorRT export is available for a further detector speed-up on NVIDIA hardware.

3.8 Performance

On an RTX 3060 Ti with three 1080p files, one pipeline iteration processes one frame from every camera:

stage	cost
batched YOLO11s detection + per-camera ByteTrack	~40 ms per iteration
classifier over the due crops of all cameras	~10-25 ms
pipeline throughput	~11-14 iterations per second

Throughput divides across cameras: five streams on an Apple-GPU laptop measured 3.6-4.7 iterations per second. Because a file advances one of its own frames per iteration, a 30 fps clip on a slow machine plays well below real time; the demo clips are therefore normalised to a low frame rate so playback is representative. This is a laptop constraint, not a deployment one.

3.9 Accuracy

Measured on 56 hand-labelled crossings from the demo footage, comparing the split voting strategy against averaging every observation:

attribute	averaging everything	split strategy
gender	90.9 %	94.5 %
adult / child	96.4 %	100 %

The evaluation is reproducible from the repository against your own footage.

3.10 Known limits

- **Counting is the robust part; demographics are the weaker part.** Counting needs the camera to see heads cleanly across the line, so an overhead or high-angle entrance camera works best and a camera looking down a corridor is much harder.
- **Zero-shot classification on full-body CCTV crops is a baseline**, not a purpose-trained attribute model, and will not match one.
- **Distant people are counted but not classified** until they come closer than the minimum crop height. They are reported as unknown rather than guessed.
- **Per-site configuration is deliberately minimal:** the trigger line, and optionally the detector confidence and minimum crop height. One pipeline runs everywhere, so there is no per-site model to retrain or to drift.